

AWS re:Invent

NOV. 28 – DEC. 2, 2022 | LAS VEGAS, NV

SVS301-R

Refactoring to serverless

Gregor Hohpe

Enterprise Strategist
AWS

Sindhu Pillai

Regional SA Lead, EdTech
AWS



© 2022, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Cloud
– Automation
= Just another data center

An opinionated architect

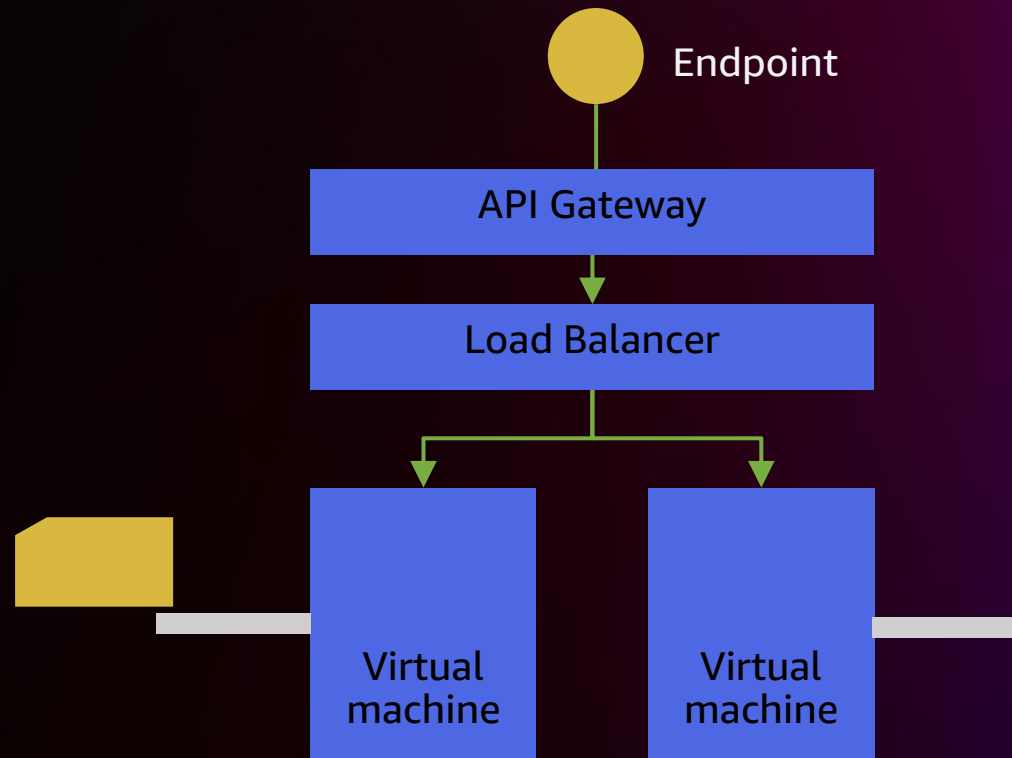
Decomposing automation

Config

Compose

Deploy

Provision



Automation languages

Document-oriented
(JSON/YAML)

Object-oriented/procedural
(Java, Python, TypeScript, etc.)

Functional
(Haskell, Clojure, F#)

```
Resources:
  S3Bucket:
    Type: 'AWS::S3::Bucket'
    Description: Amazon S3 bucket
    Properties:
      BucketName: some-bucket-name
      VersioningConfiguration:
        Status: Enabled
      BucketEncryption:
        ServerSideEncryptionConfiguration:
          - ServerSideEncryptionByDefault:
              SSEAlgorithm: AES256
```

CloudFormation
TerraForm

```
class SqsSnsStack extends cdk.Stack {
  constructor(scope: cdk.Construct, id: string,
              props?: cdk.StackProps) {
    super(scope, id, props);

    const queue =
      new sqs.Queue(this, "MySqsQueue", {
        visibilityTimeout: Duration.minutes(5);
      });

    const topic =
      new sns.Topic(this, "MySnsTopic");
    topic.addSubscription(
      new SqsSubscription(queue));
  }
}
```

AWS CDK
Pulumi

```
(ns hello-world.main
  (:require ["@pulumi/pulumi" :as pulumi]
            ["@pulumi/aws" :as aws]
            [pulumi-cljs.core :as p]))

(defn ^:export stack
  "Create the Pulumi stack"
  []
  (let [bucket
        (p/resource aws/s3.Bucket "test-bkt" nil
          {:bucket (p/cfg "bucket-name")})]

    (p/prepare-output {:bucket bucket})))
```

Pulumi with F#,
BCL (prop), CUE



AWS CDK = Infrastructure as actual code



Familiar
Your language
Classes and methods



Abstraction
Sane defaults
Reusable classes



Tool support
AutoComplete
Inline documentation

```
const vpc = new ec2.Vpc(this, 'vpc', { maxAzs: 2 });
const cluster = new ecs.Cluster(this, 'cluster', { vpc: vpc });
const service = new patterns.NetworkLoadBalancedFargateService(this, 'sample-app', {
  cluster,
  taskImageOptions: {
    image: ecs.ContainerImage.fromAsset('ping'),
  },
  dom
});
```

domainName
domainZone

(property) patterns.NetworkLoadBalancedServiceBaseProps.domainName?: string | undefined

The domain name for the service, e.g.
"api.example.com."

@default

- No domain name.



Java



Much more than

actual

Infrastructure as code

**“Cloud automation blurs the line
between application code and
automation code.”**

A modern cloud automation architect

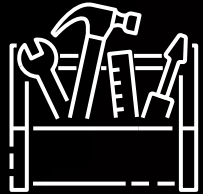


Refactoring

“A controlled technique for improving the design of an existing code base through behavior-preserving transformations.”

Martin Fowler, refactoring.com

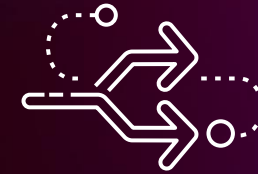
Motivations for serverless refactoring



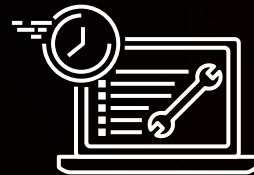
**Use platform
capability**



**AWS capabilities
evolve**



**Application
needs evolve**



**Improve runtime
characteristics**



Reduce cost

An example: Extract send message

Refactoring: Extract send message

Original

Application code (AWS Lambda)

```
exports.handler = async (event) => {  
  results = //some logic  
  const params = {  
    QueueUrl: process.env.ORDER_QUEUE_URL,  
    MessageBody: results  
  };  
  await sqs.sendMessage(params).promise();  
}
```

- ✗ Topology hidden in environment variable
- ✗ Mix of logic and dependencies in application code

Refactored

Application code (AWS Lambda)

```
exports.handler = async (event) => {  
  results = // some logic  
  return {message: results};  
};
```

Automation code (AWS CDK)

```
new Function(this, 'orderPizza', {  
  functionName: `OrderPizza`,  
  runtime: Runtime.NODEJS_14_X,  
  code: Code.fromAsset('..'),  
  handler: 'orderPizza.handler',  
  onSuccess: new SqsDestination(this.pizzaQueue)  
});
```

- ✓ No external dependencies in application code
- ✓ Topology defined in automation code
- ✓ Automated permission grant

Did the refactoring preserve behavior?

Only works with
asynchronous function
invocation

Result is wrapped
in an event envelope

Invocation occurs at
the end of function
execution

SQS message using code

Message: 82f7e789-1de8-4019-bf50-9cccee500e83		
Details	Body	Attributes
	<code>{"action": "build_pizza", "type": "Cheese"}</code>	

SQS message using Lambda destination

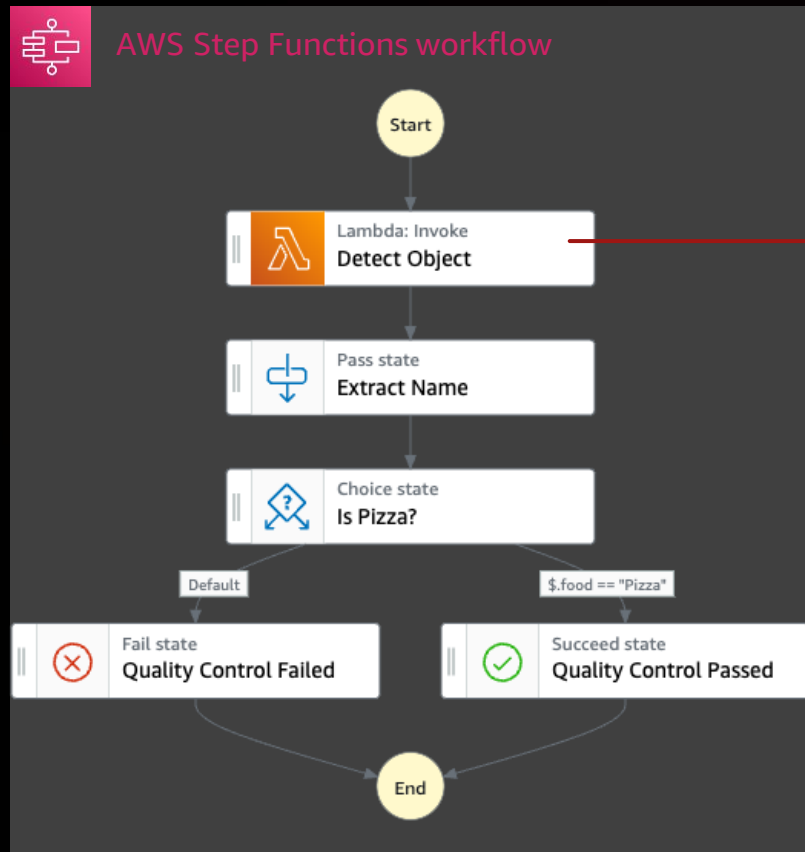
Message: f98b88d1-e653-45b9-b6aa-c3c25ce0e1b6		
Details	Body	Attributes
	<pre>{ "version": "1.0", "timestamp": "2022-10-04T23:21:53.126Z", "requestContext": { "requestId": "db129211-3869-4a16-922c-c0559bb92b4b", "functionArn": "arn:aws:lambda:us-west-2:123456789012:function:bakePizza_destination:\$LATEST", "condition": "Success", "approximateInvokeCount": 1 }, "requestPayload": {}, "responseContext": { "statusCode": 200, "executedVersion": "\$LATEST" }, "responsePayload": { "action": "build_pizza", "type": "Cheese" } }</pre>	

Let's brainstorm some patterns

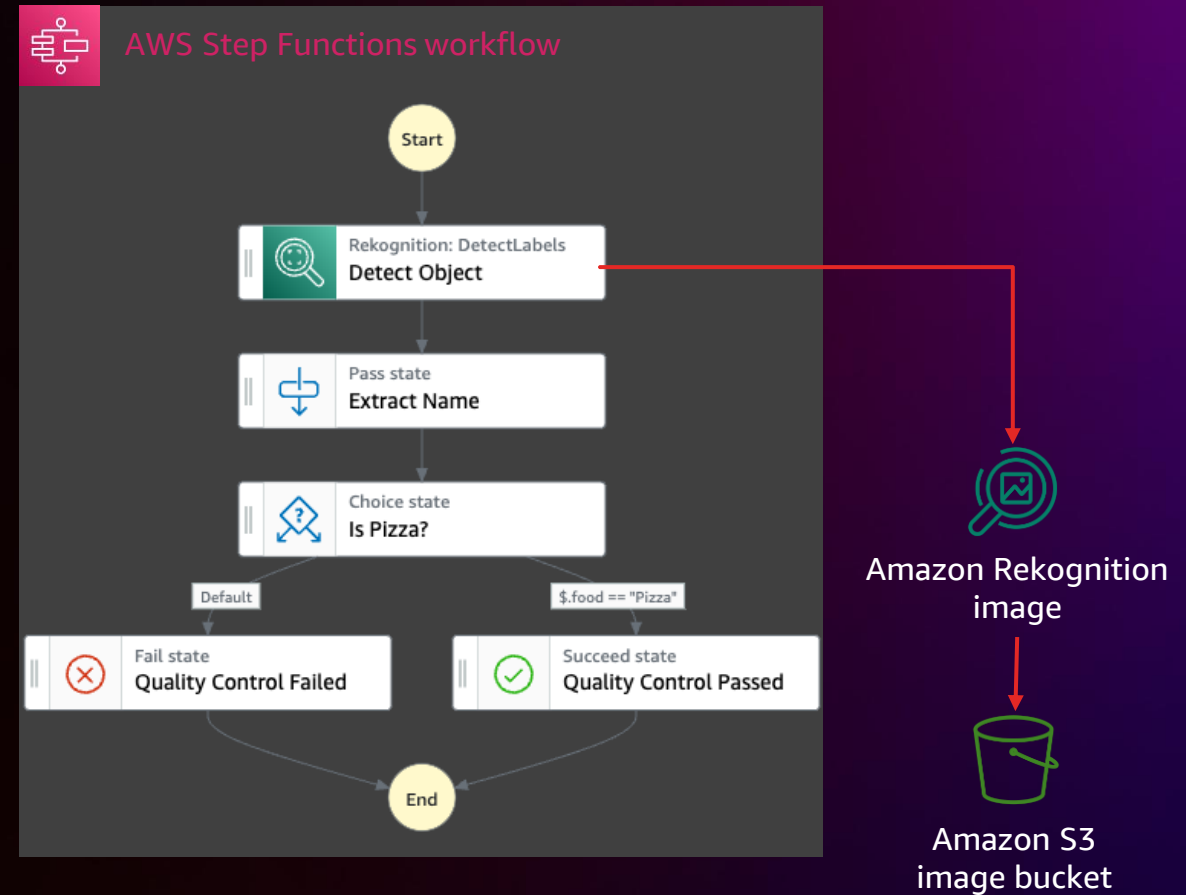
An example: Replace Lambda with service integration

Refactoring: Replace Lambda with service integration

Original



Refactored



Original

Automation code (AWS CDK)

```
const detectObjectInImageLambda = new lambda.Function(this,
'detectObjectInImage', {
  functionName: 'detectObjectInImage',
  runtime: lambda.Runtime.NODEJS_14_X,
  code: lambda.Code.fromAsset('lambda-fns'),
  handler: 'detectObjectInImage.handler'
});

const rekognitionPolicy = new iam.PolicyStatement({
  actions: ["s3:GetObject", "rekognition:DetectLabels"],
  resources: ['*'] //rekognition requires 'all' })

detectObjectInImageLambda.addToRolePolicy(rekognitionPolicy)

const detectObject = new tasks.LambdaInvoke(this, 'Detect', {
  lambdaFunction: detectObjectInImageLambda,
  payload: sfm.TaskInput.fromObject({
    s3Bucket: imageBucket.bucketName,
    imageName: this.IMAGE_TO_LABEL
  }),
  outputPath: "$.Payload"
});
```

Application code (AWS Lambda)

```
exports.handler = async (event) => {
  var params = {
    Image: {
      S3Object: {
        Bucket: event.bucketName,
        Name: event.imageName
      }
    }
  };
  const result = await
  rekognition.detectLabels(params).promise();
  callback(null, result);
}
```

Refactored

Automation code (AWS CDK)

```
const detectObject = new tasks.CallAwsService(this, 'Detect', {
  service: 'rekognition',
  action: 'detectLabels',
  parameters: {
    Image: {
      S3Object: {
        Bucket: imageBucket.bucketName,
        Name: this.IMAGE_TO_LABEL
      }
    }
  },
  iamResources: ['*'],
  additionalIamStatements: [
    new iam.PolicyStatement({
      actions: ['s3:getObject'],
      resources:
        [`${imageBucket.bucketArn}/${this.IMAGE_TO_LABEL}`]
    })
  ]
});
```



- ✓ No application code to maintain
- ✓ Topology defined in automation code

✗ Application code just for service integration



More refactoring patterns

Use platform capability	AWS capabilities evolve	Application needs evolve	Improve runtime characteristics
Pattern name	Description		
Extract Send Message	Instead of sending Amazon SQS messages via code, use AWS Lambda Destinations instead		
Extract Function Invocation	Instead calling one AWS Lambda function directly from another, use Lambda Destinations instead		
Extract Message Filter	Eliminate invalid messages with Amazon EventBridge instead of conditional statements at the consumer end		

More refactoring patterns

Use platform
capability

AWS capabilities evolve

Application
needs evolve

Improve runtime
characteristics

Pattern name	Description
Replace Lambda with service integration	Service integration allows direct calls to any API from AWS Step Functions without the need for an additional Lambda function
Direct database access	Replace a Lambda function that only reads from Amazon DynamoDB with the Step Functions <code>getItem</code> task

More refactoring patterns

**Use platform
capability**

AWS capabilities evolve

**Application
needs evolve**

**Improve runtime
characteristics**

Pattern name	Description
Replace event pattern with Lambda	If an event pattern can no longer be implemented in Amazon EventBridge, build it in Lambda instead

More refactoring patterns

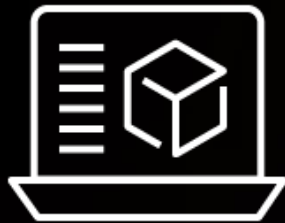
Use platform capability	AWS capabilities evolve	Application needs evolve	Improve runtime characteristics
Pattern name		Description	
Replace polling with 'Wait for Callback'		Instead of polling for results, use AWS Step Functions 'Wait for Callback' to pause the workflow until a task token is returned	
Replace map with Scatter-Gather		Instead of making parallel invocations from a Step Functions map state, send a message to Amazon SNS	
Convert choreography to orchestration		Replace message flow with central workflow	

**“Cloud automation isn’t an afterthought.
It impacts your architecture choices and
blurs the lines between application and
platform code.”**

A modern cloud automation application architect

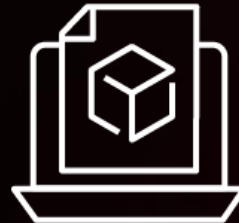
Continue your AWS Serverless learning

Learn at your
own pace



Expand your Serverless skills with our Learning Plan on **AWS Skill Builder**

Increase your
knowledge



Use our **Ramp-Up Guides** to build your Serverless knowledge

Earn AWS
Serverless badge



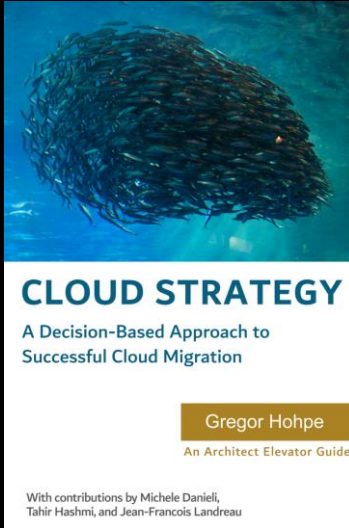
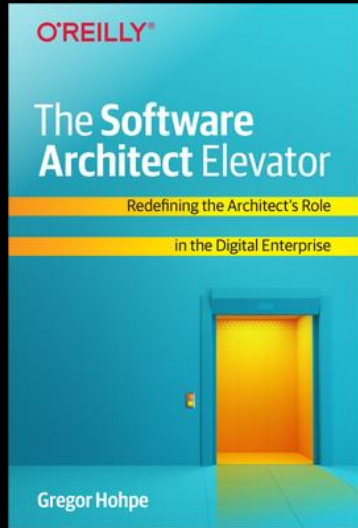
Demonstrate your Knowledge by achieving **digital badges**



<https://s12d.com/serverless-learning>



Want to learn more?



serverlessland.com/reinvent2022/svs301

Welcome to Serverless Land

This site brings together all the latest blogs, videos, and training for AWS Serverless. Learn to use and build apps that scale automatically on low-cost, fully-managed serverless architecture.

[Learn More](#)



Gregor Hohpe

@ghohpe

www.linkedin.com/in/ghohpe

ArchitectElevator.com

Sindhu Pillai

www.linkedin.com/in/sindhu-pillai-27b54a16/



© 2022, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Thank you!

Gregor Hohpe

hohpe@amazon.com

[@ghohpe](#)

[Linkedin.com/in/ghohpe](https://www.linkedin.com/in/ghohpe)

Sindhu Pillai

pisindhu@amazon.com



Please complete the session survey in the **mobile app**



© 2022, Amazon Web Services, Inc. or its affiliates. All rights reserved.